The central step of this procedure is to define the log likelihood function as well as a gradient descent algorithm. As mentioned above, the former is an instance of the `gpytorch.mlls.ExactMarginalLogLikelihood` class, implemented like so:

```
mll = gpytorch.mlls.ExactMarginalLogLikelihood(likelihood, model)
```

For the gradient descent algorithm, we use Adam, which is the state-of-the-art algorithm that has enjoyed a lot of success in many machine learning tasks, especially deep learning. We declare it using PyTorch as follows:

```
optimizer = torch.optim.Adam([model.mean_module.constant], lr=0.01)
```

Note that we are passing to the `torch.optim.Adam` class `model.mean_module.constant`, which is the mean value that we seek to optimize. When we run the gradient descent procedure, the Adam algorithm will iteratively update the value of `model.mean_module.constant` to improve the likelihood function.

The last thing we need to do now is to run gradient descent, which is implemented below.

```
model.train()  ❶
likelihood.train()  ❶

losses = []
constants = []
for i in tqdm(range(500)):
    optimizer.zero_grad()

    output = model(train_x)
    loss = -mll(output, train_y)  ❷

    loss.backward()  ❸

    losses.append(loss.item())
    constants.append(model.mean_module.constant.item())

    optimizer.step()  ❸

model.eval()  ❹
likelihood.eval()  ❹
```

❶ Enable training model.

❷ Loss as the negative marginal log likelihood.

❸ Gradient descent on the loss.

❹ Enable prediction model.

The calls to `train()` at the beginning and to `eval()` at the end are the bookkeeping that we always need to do, which respectively enables training mode and prediction mode of our GP model. Resetting the gradients at each step with `optimizer.zero_grad()` is another bookkeeping task to make sure we don't incorrectly compute the gradients.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>